

# SKU Economic Health Intelligence System

## Technical Documentation & System Guide

Excel + Google Apps Script Architecture

Version 1.0 | February 2026

### 1. System Overview

The SKU Economic Health Intelligence System is a multi-agent automation framework built inside Excel (Google Sheets) and powered by Google Apps Script. Its purpose is to automatically monitor the financial health and performance of every Amazon SKU, identify root causes of decline, and issue prioritised action cards to the responsible Product Holder (PH).

Each SKU passes through a sequential pipeline of five intelligent agents. Every agent writes its results to a dedicated sheet, and the final output is a Mission Card that tells the PH exactly what to do and by when.

#### 1.1 End-to-End Pipeline

| Stage  | Sheet                   | Agent Role                                 | Output                    |
|--------|-------------------------|--------------------------------------------|---------------------------|
| Input  | RAW_INPUT_DATA          | Raw Amazon performance data feed           | 40 metric columns per SKU |
| Step 1 | Agent0_Data_Guard       | Data quality & sufficiency gating          | PASS / FAIL flag + score  |
| Step 2 | Agent1_State_Sentinel   | Assign health state to each SKU            | Green / Amber / Red state |
| Step 3 | State_History           | Persist state across runs                  | Historical state log      |
| Step 4 | Agent2_Driver_Analyst   | Root-cause analysis                        | Primary driver identified |
| Step 5 | Agent3_Cadence_Control  | Set monitoring frequency & escalation flag | Next review date          |
| Step 6 | Agent4_Mission_Selector | 5-Gate decision engine                     | OPEN/ HOLD / REJECT       |

| Stage  | Sheet            | Agent Role                        | Output                |
|--------|------------------|-----------------------------------|-----------------------|
| Output | PH_Mission_Cards | Actionable mission card per SKU   | Assigned to PH Holder |
| Ref    | Gate_Pass_Fail   | Static gate rules reference table | Pass / Fail criteria  |

## 2. RAW\_INPUT\_DATA Sheet

This is the source-of-truth input sheet. It contains one row per SKU and spans 40 columns (A–AN) covering identity, revenue, traffic, advertising, fulfilment, and financial summary metrics exported from Amazon Seller Central.

### 2.1 Identity Columns (A–E)

| Col | Field     | Description                                               |
|-----|-----------|-----------------------------------------------------------|
| A   | SKU_ID    | Sequential unique number identifying each SKU row         |
| B   | SKU_Name  | Product variant code (e.g., WCB7BS2PK+RPR44WH2PK)         |
| C   | ASIN      | Amazon Standard Identification Number                     |
| D   | PH_Holder | Product Holder — the team member responsible for this SKU |
| E   | Account   | Amazon Seller Account name the SKU belongs to             |

### 2.2 NNR% — Net New Revenue Percentage (F–I)

NNR% is the core profitability signal used by Agent 1 to classify SKU health. It measures what percentage of revenue is actual net profit after all costs.

**Formula:**  $\text{NNR\%} = (\text{Total Income} - \text{Total Expenses}) / \text{Total Income} \times 100$

A positive NNR% means the SKU is profitable; a negative NNR% means it is losing money on every sale.

| Col | Field    | Period        | Purpose in System                                                    |
|-----|----------|---------------|----------------------------------------------------------------------|
| F   | NNR%_3M  | Last 3 months | Long-term trend baseline — used as 'Previous' anchor in Agent 1      |
| G   | NNR%_30D | Last 30 days  | Primary state signal — determines Red / Amber / Green classification |
| H   | NNR%_10D | Last 10 days  | Short-term confirmation of current trend                             |

| Co I | Field   | Period      | Purpose in System                                              |
|------|---------|-------------|----------------------------------------------------------------|
| I    | NNR%_5D | Last 5 days | Most recent signal — used to confirm or contradict 30D reading |

## 2.3 Orders Count (J–M)

Order counts are used by Agent 0 to assess whether there is sufficient sales volume to make the data statistically meaningful. Low order counts result in score deductions.

| Co I | Field      | Period              | Agent 0 Threshold     |
|------|------------|---------------------|-----------------------|
| J    | Orders_3M  | 3-month order count | < 30 orders → -10 pts |
| K    | Orders_30D | 30-day order count  | < 10 orders → -10 pts |
| L    | Orders_10D | 10-day order count  | < 5 orders → -5 pts   |
| M    | Orders_5D  | 5-day order count   | < 3 orders → -5 pts   |

## 2.4 Revenue (N–Q)

| Co I | Field       | Description                     |
|------|-------------|---------------------------------|
| N    | Revenue_3M  | Gross revenue over 3 months (£) |
| O    | Revenue_30D | Gross revenue over 30 days (£)  |
| P    | Revenue_10D | Gross revenue over 10 days (£)  |
| Q    | Revenue_5D  | Gross revenue over 5 days (£)   |

## 2.5 NNR in £ (R–S)

| Co I | Field     | Description                                      |
|------|-----------|--------------------------------------------------|
| R    | NNR_£_3M  | Net New Revenue in absolute pounds over 3 months |
| S    | NNR_£_30D | Net New Revenue in absolute pounds over 30 days  |

## 2.6 Traffic & Conversion (T–V)

| Co I | Field        | Formula / Description                          |
|------|--------------|------------------------------------------------|
| T    | Sessions_3M  | Total Amazon listing page visits over 3 months |
| U    | Sessions_30D | Total Amazon listing page visits over 30 days  |

| Co I | Field    | Formula / Description                                                           |
|------|----------|---------------------------------------------------------------------------------|
| V    | CVR%_30D | Conversion Rate 30D = (Orders_30D / Sessions_30D) × 100 — % of visitors who buy |

## 2.7 PPC / Advertising Metrics (W–Y)

| Co I | Field         | Formula / Description                                                      |
|------|---------------|----------------------------------------------------------------------------|
| W    | PPC_Spend_30D | Total Amazon PPC advertising spend in last 30 days (£)                     |
| X    | PPC_Sales_30D | Total sales attributed to PPC ads in last 30 days (£)                      |
| Y    | ACoS%_30D     | Advertising Cost of Sale = (PPC_Spend / PPC_Sales) × 100 — lower is better |

## 2.8 Returns, Postage & Buy Box (Z–AE)

| Co I | Field              | Description                                                |
|------|--------------------|------------------------------------------------------------|
| Z    | Return%_30D        | Customer return rate over 30 days (%)                      |
| AA   | Postage%_Income_3M | Postage cost as a percentage of total income over 3 months |
| AE   | BuyBox%_30D        | Amazon Buy Box win rate over 30 days — ideally 100%        |

## 2.9 Financial P&L Summary (AB–AD)

| Co I | Field            | Formula / Description                                                    |
|------|------------------|--------------------------------------------------------------------------|
| AB   | TotalIncome_3M   | Total revenue plus all other income streams over 3 months (£)            |
| AC   | TotalExpenses_3M | Total expenses: COGS + Amazon fees + postage + PPC over 3 months (£)     |
| AD   | Margin%_3M       | Actual profit margin = (TotalIncome – TotalExpenses) / TotalIncome × 100 |

## 2.10 Stock, Dates & Extended Metrics (AF–AN)

| Co I | Field             | Description                                                 |
|------|-------------------|-------------------------------------------------------------|
| AF   | Stock_DaysOfCover | How many days current stock will last at current sales rate |

| Co I | Field             | Description                                                     |
|------|-------------------|-----------------------------------------------------------------|
| AG   | Data_LastDate     | The latest date covered by the data in this row                 |
| AH   | Periods_Reference | Human-readable date ranges for each analysis window             |
| AI   | Scenario          | Same as Periods_Reference — passed to agents as context         |
| AJ   | CVR%_3M           | Conversion Rate averaged over 3 months                          |
| AK   | ACoS%_3M          | ACoS averaged over 3 months                                     |
| AL   | Return%_3M        | Return rate averaged over 3 months                              |
| AM   | BuyBox%_3M        | Buy Box win rate averaged over 3 months                         |
| AN   | Postage%_Income   | Postage as % of income (same as AA, passed for agent reference) |

## 3. Agent 0 — Data Guard

Agent 0 is the gatekeeper of the pipeline. Before any analysis begins, it checks whether each SKU has sufficient, clean, and reliable data. If a SKU fails this gate, it is excluded from all downstream agents.

### 3.1 Data Sufficiency Score

Each SKU starts with a score of 100. Points are deducted based on data gaps:

| Check           | Condition          | Deduction | Rationale                                                |
|-----------------|--------------------|-----------|----------------------------------------------------------|
| Orders_3M gate  | < 30 orders in 3M  | −10 pts   | Insufficient long-term volume for statistical confidence |
| Orders_30D gate | < 10 orders in 30D | −10 pts   | Insufficient short-term volume                           |
| Orders_10D gate | < 5 orders in 10D  | −5 pts    | Sparse recent data                                       |
| Orders_5D gate  | < 3 orders in 5D   | −5 pts    | Very sparse recent data                                  |
| Null fields     | Each missing field | −2 pts    | Each null column reduces reliability                     |

### Score Category Thresholds:

| Score Range      | Category | Meaning                                                                                  |
|------------------|----------|------------------------------------------------------------------------------------------|
| 71–100           | Rich     | Good data with strong coverage — all calculations are fully reliable                     |
| 40–70            | Adequate | Minimum passing threshold — results are usable with caveats                              |
| < 40             | Thin     | Sparse data — WARN if score 32–39; FAIL if score < 32. Thin data cap applied by Agent 1. |
| 0 (corrupt data) | FAIL     | Data is corrupt or critically invalid — SKU is excluded from this pipeline run           |

## 3.2 Additional Quality Checks

| Column | Field             | Logic                                                                                                                             |
|--------|-------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| O      | OutlierFlag       | TRUE if Revenue or NNR% values are statistical outliers (e.g. extreme spikes)                                                     |
| Q      | SystemicIssueFlag | TRUE if multiple SKUs show identical zero patterns — suggests a broken data feed                                                  |
| H      | DataQualityFlag   | OK or CORRUPT based on data integrity checks                                                                                      |
| R      | Agent0_Decision   | Final gate output: PASS (score $\geq$ 40, no systemic issue), WARN (score 32–39, thin data), or FAIL (score < 32 or corrupt data) |
| S      | DataIssuesSummary | Human-readable explanation of all deductions made (e.g. 'Orders_3M<30 (-10)')                                                     |

# 4. Agent 1 — State Sentinel

Agent 1 assigns a health state to each SKU that passed Agent 0. It works like a traffic light system, using NNR%\_30D as the primary signal and NNR%\_3M as the trend context.

## 4.1 State Classification Logic

| State       | NNR%_30D Condition | Meaning                          | Urgency            |
|-------------|--------------------|----------------------------------|--------------------|
| Dark_Green  | > 15%              | Excellent — strong profitability | No action required |
| Light_Green | 5% to 15%          | Good — stable and healthy        | Monitor only       |

| State     | NNR%_30D Condition | Meaning                                 | Urgency              |
|-----------|--------------------|-----------------------------------------|----------------------|
| Amber     | 0% to 5%           | Warning — margin declining or near-zero | Investigation needed |
| Red       | < 0%               | Critical — SKU is actively losing money | Immediate action     |
| Purple    | New listing        | Insufficient history to judge           | Bootstrap period     |
| Data_Risk | Data issues        | State cannot be determined reliably     | Fix data first       |

## 4.2 Hysteresis — Preventing Flip-Flop

A single bad data point should not immediately trigger an escalation. Agent 1 uses a persistence (hysteresis) mechanism: a state must be sustained over multiple consecutive runs before it is treated as confirmed.

| State           | Persistence Required          | Logic                                                        |
|-----------------|-------------------------------|--------------------------------------------------------------|
| Amber confirmed | ≥ 2 consecutive runs in Amber | Two runs with NNR%_30D between 0–5% confirms the Amber state |
| Red confirmed   | ≥ 3 consecutive runs in Red   | Three runs with negative NNR%_30D confirms the Red state     |
| Upgrade         | Immediate                     | Recovery is reflected immediately — no delay on improvement  |

## 4.3 Output Fields

| Co I | Field                  | Description                                                |
|------|------------------------|------------------------------------------------------------|
| F    | CurrentState           | Health state assigned in this run                          |
| G    | PreviousState          | Health state from the previous run                         |
| H    | StateChange            | Upgrade / Downgrade / No Change                            |
| I    | ConfidenceScore        | How confident the system is in this state (0–100)          |
| J    | SustainedPeriods_Amber | Number of consecutive runs in Amber state                  |
| K    | SustainedPeriods_Red   | Number of consecutive runs in Red state                    |
| L    | ThinDataCapActive      | TRUE if data is too sparse for full analysis               |
| M    | HysteresisActive       | TRUE if a state change was suppressed by persistence logic |
| N    | DataSufficiencyFlag    | Sufficient / Insufficient (passed from Agent 0)            |
| O    | ObserveOnly            | TRUE if SKU is in a bootstrap/observe-only period          |
| P    | Reason_Anchor          | Primary reason: current 30D NNR% with label                |

| Co I | Field           | Description                                         |
|------|-----------------|-----------------------------------------------------|
| Q    | Reason_Trend    | Trend context: 3M NNR% for comparison               |
| R    | Reason_DataNote | Combined debug note: score, flags, sustained counts |

### 4.4 State\_History Sheet

After every run, Agent 1 appends a row to the State\_History sheet. This log is the foundation for Agent 1's persistence checks — it reads back the history to count how many consecutive runs a SKU has been in a given state.

| Co I | Field        | Description                                                  |
|------|--------------|--------------------------------------------------------------|
| A    | Run_ID       | The run identifier for that entry (e.g., RUN-2026-02-26-001) |
| B    | SKU_ID       | Reference to the SKU                                         |
| C    | CurrentState | State recorded in that run                                   |
| D    | NNR_30D      | 30D NNR% at the time of that run                             |
| E    | NNR_10D      | 10D NNR% at the time of that run                             |
| F    | SP_Amber     | Amber sustained period count at time of that run             |
| G    | SP_Red       | Red sustained period count at time of that run               |
| H    | Timestamp    | Date the run was executed                                    |

---

## 5. Agent 2 — Driver Analyst

---

Agent 2 answers the question: WHY is this SKU in Amber or Red? It analyses the available metrics to identify the primary root cause (driver) and classifies whether it is structural (permanent), temporary, or mixed.

### 5.1 Driver Categories

| Category    | Specific Drivers                                  | Triggered By                                             |
|-------------|---------------------------------------------------|----------------------------------------------------------|
| MARGIN_COST | unit_margin_compression, fulfilment_cost_increase | High postage%, low Margin%_3M, rising expenses vs income |
| TRAFFIC     | traffic_drop, suppressed_listings                 | Sessions falling, low or zero Sessions_30D               |
| CONVERSION  | cvr_decline, poor_buybox                          | CVR% falling, BuyBox% below threshold                    |



| Category    | Specific Drivers         | Triggered By                                           |
|-------------|--------------------------|--------------------------------------------------------|
| ADVERTISING | high_acos, ppc_waste     | ACoS% high relative to NNR%, PPC spend not converting  |
| RETURNS     | high_return_rate         | Return%_30D or Return%_3M above acceptable thresholds  |
| STOCK       | stockout_risk, overstock | Stock_DaysOfCover very low (<14D) or very high (>365D) |

## 5.2 Driver Classification

The classification determines whether the issue is worth escalating for immediate action or whether it needs more time to develop:

| Classification | Meaning                                                                                    | Gate 4 Impact                      |
|----------------|--------------------------------------------------------------------------------------------|------------------------------------|
| Structural     | Systematic, long-term issue requiring a deliberate fix — e.g. COGS, pricing, fee structure | Gate 4 PASS — escalation warranted |
| Temporary      | Short-term dip — seasonal lull, one-off stock event, data anomaly                          | Gate 4 FAIL — wait and monitor     |
| Mixed          | Combination of structural and temporary factors — signal is ambiguous                      | Gate 4 FAIL — conditional at best  |

## 5.3 Driver Confidence Score

After a driver is identified, a DriverConfidenceScore (0–100) is calculated. A score of  $\geq 70$  is required for Gate 4 to pass in Agent 4.

- 80–90: Multiple independent signals all point to the same driver
- 60–75: A single strong signal indicates the driver
- < 60: Conflicting or ambiguous signals — low confidence

## 5.4 Output Fields

| Col | Field                 | Description                                                    |
|-----|-----------------------|----------------------------------------------------------------|
| E   | PrimaryDriver         | The main root cause identified (e.g., unit_margin_compression) |
| F   | DriverCategory        | Category group: MARGIN_COST, TRAFFIC, CONVERSION, etc.         |
| G   | DriverClassification  | Structural / Temporary / Mixed                                 |
| H   | DriverConfidenceScore | Confidence in the driver identification (0–100)                |
| I   | SecondaryDrivers      | Additional contributing factors identified                     |

| Co I | Field                 | Description                               |
|------|-----------------------|-------------------------------------------|
| J    | ExplanationConfidence | Overall explanation confidence score      |
| K    | DriverNotes           | Human-readable summary of driver analysis |

## 6. Agent 3 — Cadence Control

Agent 3 determines how frequently each SKU should be reviewed and whether escalation should be allowed to proceed to Agent 4's Gate 5. It prevents unnecessary escalations for short-lived issues while ensuring critical SKUs are reviewed frequently.

### 6.1 Monitoring Cadence Logic

| SKU State   | Driver Type | Cadence | Next Review In                    |
|-------------|-------------|---------|-----------------------------------|
| Red         | Structural  | 5D      | 5 days — aggressive monitoring    |
| Red         | Temporary   | 5D      | 5 days — watch for recovery       |
| Amber       | Structural  | 5D      | 5 days — close watch              |
| Amber       | Temporary   | 5D      | 5 days — monitor trend            |
| Light_Green | Any         | 14D     | 14 days — routine bi-weekly check |
| Dark_Green  | Any         | 30D     | 14 days — bi-weekly monitoring    |

### 6.2 Escalation Allowance Rules

EscalationAllowed controls Gate 5 in Agent 4. It is set to TRUE only when specific conditions are met:

| Condition                                                             | EscalationAllowed | Reason                                           |
|-----------------------------------------------------------------------|-------------------|--------------------------------------------------|
| Structural driver AND DriverConf $\geq$ 70 AND CooldownActive = FALSE | TRUE              | Issue is real and ready for action               |
| Driver is Temporary                                                   | FALSE             | Issue may self-resolve — wait and watch          |
| CooldownActive = TRUE                                                 | FALSE             | State is flip-flopping — escalation is premature |
| StateConfidence < 70                                                  | FALSE             | System is not confident enough in the state      |

## 6.3 Output Fields

| Co I | Field                 | Description                                                              |
|------|-----------------------|--------------------------------------------------------------------------|
| F    | MonitoringCadence     | Review frequency: 5D / 14D / 30D                                         |
| G    | CooldownActive        | TRUE if a flip-flop pattern has been detected                            |
| H    | EscalationAllowed     | TRUE / FALSE — feeds directly into Agent 4 Gate 5                        |
| I    | EscalationBlockReason | Why escalation was blocked (e.g., 'Temporary driver', 'Confidence < 70') |
| J    | CadenceConfidence     | Low / Medium / High confidence in the cadence decision                   |
| K    | NextReviewDate        | Calculated date when this SKU should next be reviewed                    |

---

## 7. Agent 4 — Mission Selector (5-Gate Engine)

---

Agent 4 is the final decision-maker. Every SKU must pass through five sequential gates. If any gate fails, the mission is either HELD or REJECTED. Only SKUs that pass all five gates receive an APPROVED mission assigned to their PH Holder.

### 7.1 The Five Gates

| Gate   | Name               | PASS Condition                                       | FAIL Condition                                | Decision if FAIL |
|--------|--------------------|------------------------------------------------------|-----------------------------------------------|------------------|
| Gate 1 | Data Quality       | Score $\geq$ 40, no systemic issue, data not corrupt | Score < 40 OR data corrupt OR systemic freeze | REJECT           |
| Gate 2 | State Confirmed    | CurrentState = Amber or Red                          | State = Purple, Green, or Data_Risk           | REJECT           |
| Gate 3 | Persistence        | Amber: SP_Amber $\geq$ 2;<br>Red: SP_Red $\geq$ 3    | Sustained periods not yet met                 | HOLD             |
| Gate 4 | Driver Identified  | Structural driver AND DriverConfidence $\geq$ 70     | Driver is Temporary or Mixed OR conf < 70     | HOLD             |
| Gate 5 | Escalation Allowed | CooldownActive = FALSE AND StateConfidence $\geq$ 70 | Cooldown is ON or StateConfidence < 70        | HOLD             |

### 7.2 Mission Decisions

| Decision | Gate Outcome         | Meaning                                                                      |
|----------|----------------------|------------------------------------------------------------------------------|
| APPROVE  | All 5 gates PASS     | A mission card is created and assigned to the PH Holder for immediate action |
| HOLD     | Gate 3, 4, or 5 FAIL | Problem is real but not yet ready for escalation — continue monitoring       |
| REJECT   | Gate 1 or 2 FAIL     | No valid problem detected, or data is invalid — no action taken              |

## 7.3 Mission Types

| Mission Type | State + Driver     | Deadline | Required Action                                     |
|--------------|--------------------|----------|-----------------------------------------------------|
| RED_STRIKE   | Red + Structural   | 1 day    | Immediately review COGS, pricing, and fee structure |
| AMBER_FIX    | Amber + Structural | 3 days   | Investigate and fix the identified root cause       |
| AMBER_WATCH  | Amber + Mixed      | 7 days   | Monitor closely and prepare an action plan          |
| N/A          | Green or REJECT    | None     | No action required — SKU is healthy or rejected     |

## 7.4 Mission Confidence Score

$\text{MissionConfidence} = \min(\text{StateConfidence}, \text{DriverConfidence})$

*The mission confidence is the lower of the two input confidence scores. This conservatively reflects the weakest link in the analysis chain. Example: StateConfidence = 65, DriverConfidence = 80 → MissionConfidence = 65.*

# 8. PH\_Mission\_Cards Sheet (System Output)

This is the final output of the pipeline. One Mission Card is generated per SKU per run. It consolidates the analysis from all five agents into a single actionable record for the Product Holder.

| Col | Field      | Description                                                |
|-----|------------|------------------------------------------------------------|
| A   | Run_ID     | Unique run identifier (e.g., RUN-2026-02-27-001)           |
| B   | Mission_ID | Sequential mission reference number (e.g., MSN-2026-00001) |
| C   | PH_Holder  | The team member who owns this SKU and must take action     |

| Col | Field              | Description                                                   |
|-----|--------------------|---------------------------------------------------------------|
| D–F | SKU identity       | SKU_ID, SKU_Name, ASIN                                        |
| G   | MissionType        | RED_STRIKE / AMBER_FIX / AMBER_WATCH / N/A                    |
| H   | CurrentState       | Health state at the time of this run                          |
| I   | StateConfidence    | Confidence score for the state classification (0–100)         |
| J   | PrimaryDriver      | Identified root cause (e.g., unit_margin_compression)         |
| K   | DriverType         | Structural / Temporary / Mixed                                |
| L   | DriverConfidence   | Confidence score for the driver identification (0–100)        |
| M   | SecondaryDriver    | Additional contributing factors (comma-separated)             |
| N   | DataQuality        | OK or FAIL — from Agent 0                                     |
| O   | MissionConfidence  | Overall mission confidence = min(StateConf, DriverConf)       |
| P   | ActionRequired     | Specific action instruction with emoji urgency indicator      |
| Q   | StateReasonSummary | Full narrative: anchor NNR%, trend NNR%, driver, confidence   |
| R   | Owner              | Mirrors PH_Holder — the accountable person                    |
| S   | Deadline           | Date by which the PH must take action                         |
| T   | NextReviewDate     | Date of the next scheduled monitoring review                  |
| U   | MissionBlockReason | For HOLD/REJECT: the gate that caused the block (e.g., Gate5) |

---

## 9. Google Apps Script Implementation

---

The entire pipeline is automated using Google Apps Script. Each agent corresponds to a function that reads from RAW\_INPUT\_DATA and prior agent sheets, then writes its results to the appropriate output sheet.

### 9.1 Script Architecture

| Function          | Writes To | Depends On                |
|-------------------|-----------|---------------------------|
| generateRunId()   | —         | Called first in every run |
| getRawInputData() | —         | Reads RAW_INPUT_DATA      |

| Function                    | Writes To                                     | Depends On                |
|-----------------------------|-----------------------------------------------|---------------------------|
| runAgent0_DataGuard()       | Agent0_Data_Guard                             | Raw data                  |
| runAgent1_StateSentinel()   | Agent1_State_Sentinel,<br>State_History       | Agent 0 output            |
| runAgent2_DriverAnalyst()   | Agent2_Driver_Analyst                         | Agent 1 output + raw data |
| runAgent3_CadenceControl()  | Agent3_Cadence_Control                        | Agent 1 + Agent 2 output  |
| runAgent4_MissionSelector() | Agent4_Mission_Selector<br>+ PH_Mission_Cards | All prior agents          |
| runFullPipeline()           | All sheets                                    | Master orchestrator       |

## 9.2 Master Pipeline Runner

```
function runFullPipeline() {
  const runId = generateRunId();           // e.g. RUN-2026-02-27-001
  const rawData = getRawInputData();       // Read all rows from RAW_INPUT_DATA

  runAgent0_DataGuard(rawData, runId);
  runAgent1_StateSentinel(rawData, runId);
  runAgent2_DriverAnalyst(rawData, runId);
  runAgent3_CadenceControl(rawData, runId);
  runAgent4_MissionSelector(rawData, runId);

  Logger.log('Pipeline complete: ' + runId);
}
```

## 9.3 Run ID Generation

```
function generateRunId() {
  const today = Utilities.formatDate(new Date(), 'UTC', 'yyyy-MM-dd');
  const sheet = SpreadsheetApp.getActiveSpreadsheet()
    .getSheetByName('Agent0_Data_Guard');
  const lastRunId = sheet.getRange(sheet.getLastRow(), 1).getValue();

  // Increment sequence if same day, else start at 001
  const seq = lastRunId.startsWith('RUN-' + today)
    ? String(parseInt(lastRunId.slice(-3)) + 1).padStart(3, '0')
    : '001';

  return 'RUN-' + today + '-' + seq;
}
```

## 9.4 Agent 0 — Data Sufficiency Score

```
function scoreDataSufficiency(sku) {
  let score = 100;
  let issues = [];

  if (sku.Orders_3M < 30) { score -= 10; issues.push('Orders_3M<30 (-10)'); }
```

```

if (sku.Orders_30D < 10) { score -= 10; issues.push('Orders_30D<10 (-10)'); }
if (sku.Orders_10D < 5) { score -= 5; issues.push('Orders_10D<5 (-5)'); }
if (sku.Orders_5D < 3) { score -= 5; issues.push('Orders_5D<3 (-5)'); }

const nullCount = countNulls(sku);
score -= nullCount * 2;
if (nullCount > 0) issues.push('Nulls: ' + nullCount + ' (-' + nullCount*2 + ')');

return {
  score: Math.max(0, score),
  category: score >= 71 ? 'Rich' :
    score >= 40 ? 'Adequate' : 'Thin',
  decision: score >= 40 ? 'PASS' : score >= 32 ? 'WARN' : 'FAIL',
  summary: issues.join(' | ')
};
}

```

## 9.5 Agent 1 — State Classification

```

function classifyState(nnr30d, history, skuId) {
  // Classify raw state from NNR%_30D
  let rawState;
  if (nnr30d > 15) rawState = 'Dark_Green';
  else if (nnr30d >= 5) rawState = 'Light_Green';
  else if (nnr30d >= 0) rawState = 'Amber';
  else rawState = 'Red';

  // Count sustained periods from State_History
  const spAmber = countConsecutive(history, skuId, 'Amber');
  const spRed = countConsecutive(history, skuId, 'Red');

  // Apply hysteresis - confirm state only if sufficiently sustained
  const confirmed =
    (rawState === 'Amber' && spAmber >= 2) ||
    (rawState === 'Red' && spRed >= 3) ||
    rawState === 'Light_Green' ||
    rawState === 'Dark_Green';

  return { state: rawState, confirmed, spAmber, spRed };
}

```

## 9.6 Agent 2 — Driver Detection

```

function detectDriver(sku) {
  // Priority-ordered driver checks
  if (sku.Postage_pct > 25 || sku.Margin_pct < 5) {
    return { driver: 'unit_margin_compression',
      category: 'MARGIN_COST',
      type: 'Structural', confidence: 80 };
  }
  if (sku.Sessions_30D === 0 || sku.Sessions_30D < sku.Sessions_3M * 0.3) {
    return { driver: 'traffic_drop',
      category: 'TRAFFIC',
      type: 'Temporary', confidence: 75 };
  }
  if (sku.CVR_30D < sku.CVR_3M * 0.7) {
    return { driver: 'cvr_decline',
      category: 'CONVERSION',

```

```

        type: 'Mixed', confidence: 70 };
    }
    // Fallback
    return { driver: 'fulfilment_cost_increase',
            category: 'MARGIN_COST',
            type: 'Temporary', confidence: 60 };
}

```

## 9.7 Agent 4 — 5-Gate Check

```

function checkAllGates(a0, a1, a2, a3) {
  const g1 = a0.score >= 40 && !a0.systemic ? 'PASS' : 'FAIL';
  const g2 = ['Amber', 'Red'].includes(a1.state) ? 'PASS' : 'FAIL';
  const g3 = (a1.state==='Amber' && a1.spAmber >= 2) ||
    (a1.state==='Red' && a1.spRed >= 3) ? 'PASS' : 'FAIL';
  const g4 = a2.type==='Structural' && a2.confidence >= 70 ? 'PASS' : 'FAIL';
  const g5 = !a3.cooldown && a1.stateConf >= 70 ? 'PASS' : 'FAIL';

  // Find first failed gate for block reason
  const gates = {Gate1:g1, Gate2:g2, Gate3:g3, Gate4:g4, Gate5:g5};
  const failedGate = Object.keys(gates).find(k => gates[k]=== 'FAIL') || null;

  if (!failedGate) return { decision:'APPROVE', blockReason: null };
  if (['Gate1', 'Gate2'].includes(failedGate))
    return { decision:'REJECT', blockReason: failedGate };
  return
    { decision:'HOLD', blockReason: failedGate };
}

```

## 9.8 Trigger Configuration

Navigate to Extensions > Apps Script > Triggers to configure automated execution:

| Trigger Type     | Recommended Settings                        | Use Case                                                 |
|------------------|---------------------------------------------|----------------------------------------------------------|
| Time-based       | Daily at 6:00 AM UTC                        | Automatic daily pipeline run before the team starts work |
| Spreadsheet open | On open event                               | Re-run when any team member opens the sheet for review   |
| Manual button    | Button in sheet linked to runFullPipeline() | On-demand run triggered by the user                      |

## 10. Example Walkthrough — SKU: WCB7BS2PK+RPR44WH2PK

The following walkthrough traces one real SKU through every stage of the pipeline to illustrate how all calculations connect.



| Stage          | Key Inputs                                                                                         | Calculation / Logic Applied                                                                 | Result                                                                                                                                     |
|----------------|----------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| RAW_INPUT_DATA | NNR%_3M=9.12%,<br>NNR%_30D=-0.58%,<br>Orders_3M=6,<br>Margin%=4.07%,<br>Postage%=20.2%             | Raw data captured — no calculation at this stage                                            | Data ready for agents                                                                                                                      |
| Agent 0        | Orders_3M=6 (<30),<br>Orders_30D=1 (<10),<br>Orders_10D=1 (<5),<br>Orders_5D=1 (<3), 4 null fields | 100 -10 -10 -5 -5 -8 = 62<br>→ Thin category, WARN decision                                 | Score=62,<br>Category=Thin,<br>Decision=WARN                                                                                               |
| Agent 1        | NNR%_30D = -0.58% (negative)                                                                       | Negative NNR → Red state.<br>3M was +9.12% → Downgrade. SP_Red=3 (sustained 3 runs)         | State=Red,<br>Confidence=65,<br>Change=Downgrade                                                                                           |
| Agent 2        | Postage%=20.2% (high),<br>Margin%=4.07% (very low)                                                 | High postage + low margin<br>→ unit_margin_compression identified as Structural driver      | Driver=unit_margin_compression,<br>Type=Structural,<br>Confidence=80                                                                       |
| Agent 3        | State=Red,<br>Driver=Structural,<br>CooldownActive=FALSE, StateConf=65                             | Structural driver → check escalation.<br>CooldownActive=FALSE but StateConf=65 < 70 → block | Cadence=5D,<br>EscalationAllowed=FALSE,<br>BlockReason=Confidence<70                                                                       |
| Agent 4        | G1=PASS,<br>G2=PASS,<br>G3=PASS (SP_Red=3≥3),<br>G4=PASS (Structural, Conf=80≥70),<br>G5=FAIL      | Gate 5 fails because Agent 3 set<br>EscalationAllowed=FALSE (StateConf=65 < 70)             | Decision=HOLD,<br>MissionType=RED_STRIKE,<br>BlockReason=Gate 5                                                                            |
| Mission Card   | HOLD decision,<br>Gate5 blocked                                                                    | Card created with HOLD status; ActionRequired shows the urgent action for when it clears    |  URGENT (1 day):<br>Review COGS, pricing, fee structure |

*Key insight: This SKU is genuinely in crisis (Red, structural margin issue) but is currently HELD because the system lacks sufficient confidence (65 < 70) to formally escalate. The PH should still be aware and preparing, but the automated mission will not fire until confidence rises above 70.*

---

## 11. Key Formulas Reference

---

| Metric                | Formula                                                                                                    | Location in System                 |
|-----------------------|------------------------------------------------------------------------------------------------------------|------------------------------------|
| NNR%                  | $(\text{Total Income} - \text{Total Expenses}) / \text{Total Income} \times 100$                           | RAW_INPUT_D<br>ATA cols F–I        |
| Margin%_3M            | $(\text{TotalIncome\_3M} - \text{TotalExpenses\_3M}) / \text{TotalIncome\_3M} \times 100$                  | RAW_INPUT_D<br>ATA col AD          |
| CVR%                  | $(\text{Orders} / \text{Sessions}) \times 100$                                                             | RAW_INPUT_D<br>ATA cols V, AJ      |
| ACoS%                 | $(\text{PPC\_Spend} / \text{PPC\_Sales}) \times 100$                                                       | RAW_INPUT_D<br>ATA cols Y, AK      |
| DataSufficiencyScore  | 100 – deductions for low order counts – (nulls × 2). Rich: ≥71, Adequate: 40–70, Thin: <40                 | Agent0 col I                       |
| State Classification  | NNR%_30D: <0=Red, 0–5%=Amber, 5–15%=Light_Green, >15%=Dark_Green, >20%=Purple (new listing or exceptional) | Agent1 col F                       |
| Persistence Check     | Amber: SP_Amber ≥ 2; Red: SP_Red ≥ 3                                                                       | Agent1 + Agent4<br>Gate 3          |
| MissionConfidence     | min(StateConfidence, DriverConfidence)                                                                     | Agent4 col N                       |
| Deadline (RED_STRIKE) | Run date + 1 day                                                                                           | PH_Mission_Cards col S             |
| Deadline (AMBER_FIX)  | Run date + 3 days                                                                                          | PH_Mission_Cards col S             |
| NextReviewDate (5D)   | Run date + 5 days                                                                                          | Agent3 +<br>PH_Mission_Cards col T |
| NextReviewDate (14D)  | Run date + 14 days                                                                                         | Agent3 +<br>PH_Mission_Cards col T |

---

## 12. Gate\_Pass\_Fail Reference Sheet

---

The Gate\_Pass\_Fail sheet is a static lookup table that defines the exact pass and fail conditions for each of the five gates. It serves as the authoritative specification for Agent 4 logic.

| Gate                    | PASS Condition                                               | FAIL Condition                                               |
|-------------------------|--------------------------------------------------------------|--------------------------------------------------------------|
| Gate1_DataQuality       | Data is OK + Sufficiency Score $\geq 40$ + No systemic issue | Data corrupt OR Score $< 40$ OR Systemic freeze detected     |
| Gate2_StateConfirmed    | State = Amber or Red                                         | State = Purple, Dark_Green, Light_Green, or Data_Risk        |
| Gate3_Persistence       | Amber $\geq 2$ sustained runs OR Red $\geq 3$ sustained runs | Sustained period threshold not met OR Gate 2 already FAIL    |
| Gate4_DriverIdentified  | Structural driver found AND DriverConfidence $\geq 70$       | No driver, OR Temporary/Mixed classification, OR Conf $< 70$ |
| Gate5_EscalationAllowed | CooldownActive = FALSE AND StateConfidence $\geq 70$         | Cooldown = TRUE (flip-flop) OR StateConfidence $< 70$        |